Compilers: Every Method with Practice, Solution and Gotcha

29 methods for the midterm. Each one: the method, a practice question stated in full, its solution worked step by step right underneath, and the trap that costs points

Compilers

2026-10-31

How to use it. For each method: read the method box; cover the green *Solution* box with a sheet of paper and do the practice question; uncover it and compare step by step; then read the red *Gotcha*. The tag on the right of each title says which mock-midterm question used the method ("course" means it is in the lectures but the mock did not test it). Every solution was checked with `check_lex.py` and `check_ll1.py`. Conventions are the textbook's: Thompson concatenation merges states (convention B), stack top on the right, productions numbered top to bottom.

Short on time? Do §§18–23 (nullable, FIRST, FOLLOW, table, trace, errors): they are Problems 3–5 of the mock, almost half the points.

Contents

1 Which phase catches the error? [course] **3**

2 Writing a regular expression for a token [mock 1.7(b), 2.3(a)] **4**

3 Thompson's construction: regex to NFA [course] **5**

4 Simulating an NFA: ε-closure and move [course] **7**

5 Subset construction: NFA to DFA [mock 1.7(a)] **8**

6 DFA minimisation by table filling [course] **9**

7 State elimination: DFA to regex [course] **10**

8 Brzozowski derivatives [course] **11**

9 Designing a DFA directly from a description [mock 2.3(b)] **12**

10 Proving a language is not regular; DFA lower bounds [mock 1.2, 2.3(c)] **13**

11 The combined scanner DFA and rule priority [mock 2.2, 2.4] **13**

12 Longest-match scanning with backup, counting reads [mock 2.1] **15**

13 Lexer states for nested comments [mock 2.3(c)] **16**

14 Writing a context-free grammar for a language [mock 1.2(b)] **17**

15 Leftmost and rightmost derivations, parse trees [mock 1.3] **19**

16 Ambiguity, precedence and associativity [mock 1.4(a)–(c)] **20**

17 Recursive descent: grammar to code and back [mock 4.1, 4.2(b), 4.5] **22**

18 Nullable nonterminals [mock 1.6(a), 5(a)] **25**

19 FIRST sets [mock 1.5(a), 3.2, 5(a)] **26**

20 FOLLOW sets [mock 1.5(a), 3.2, 5(a)] **27**

21 Building the LL(1) table and finding conflicts [mock 1.5(b), 3.3, 5(a)] **28**

22 The LL(1) stack trace [mock 4.3, 5(b)] **29**

23 Where is a syntax error caught? [mock 4.4] **30**

24 Removing immediate left recursion [mock 1.4(d), 3.1(b)] **31**

25 Removing indirect left recursion [course] **32**

26 Left factoring [course] **33**

27 Eliminating ε-productions [mock 1.6] **34**

28 Deciding "is this grammar LL(1)?" [mock 3.1(a), 5(a)] **35**

29 Classifying a grammar and its language (Chomsky) [mock 1.1] **36**

Part A. Lexical analysis

1 Which phase catches the error? [course]

Method

Ask four questions in this order and stop at the first "no".

1. **Lexical**: can the characters be cut into legal tokens by longest match? A character that starts no token, or a string or comment that never closes, is a lexical error.
2. **Syntax**: is the *sequence of tokens* allowed by the grammar?
3. **Semantic**: does it break a rule the grammar cannot express (types, declarations, number of arguments)?
4. Otherwise the program compiles; anything left is a run-time or logic error.

Practice

Given. Seven C fragments. Each contains exactly one error.

(a) `x = 5 @ 2;`
(b) `y = x + ;`
(c) `int 3x = 5;`
(d) `int a; a = "hi";`
(e) `undeclared = 1;` (never declared)
(f) `s = "abc;` (the line ends here)
(g) `if (x) else y = 1;`

Your task. For each fragment, name the phase that reports the error (lexical, syntax or semantic) and give the reason in one line.

Solution

	Phase	Reason
(a)	lexical	`@` cannot start any token of C.
(b)	syntax	every token is legal, but `+` must be followed by an operand.
(c)	syntax	longest match gives INT(3) ID(x), two legal tokens; the grammar does not allow a number where a variable name is expected.
(d)	semantic	the tokens and the structure are fine; assigning a string to an `int` is a type error.
(e)	semantic	the statement is well formed; "declared before use" is not expressible in a CFG.
(f)	lexical	the string literal is never closed, so no token can be formed.
(g)	syntax	`if (x)` must be followed by a statement before `else`.

Gotcha

(c) looks like a "bad identifier", so people answer lexical. But the lexer never knows what the grammar expects: it happily returns INT then ID. The error is found one phase later.

2 Writing a regular expression for a token [mock 1.7(b), 2.3(a)]

Method

1. Cut the lexeme into parts. For each part decide: mandatory, optional (`?`), or repeated (`*` zero or more, `+` one or more).
2. Write one sub-expression per part; use | for alternatives inside a part.
3. Check every must-match example (split it into the parts) and every must-not example (name the part that rejects it). Precedence: star binds tighter than concatenation, which binds tighter than |.

Practice

Given. Use a reduced alphabet of three symbols: the quote `"`, the backslash `\`, and x, which stands for any other character. A *string literal* is: an opening `"`; then any number of body items, where a body item is either an x or an *escape* (a `\` followed by any one of the three symbols); then a closing `"`.

Your task. (a) Write a regular expression for string literals. (b) Show how each of these is matched: `""`, `"x\"x"`, `"\\"`. (c) Explain why each of these is *not* matched: `"x`, `"\"`, `"x"x"`.

Solution

Step 1, parts. Opening quote (mandatory), body items (repeated, zero or more), closing quote (mandatory). A body item has two forms.

Step 2, expression.

$$\texttt{"}\ (\ x\ \mid\ \backslash(x \mid \texttt{"} \mid \backslash)\ )^{*}\ \texttt{"}$$

Step 3, checks. `""`: zero body items. `"x\"x"`: items x, escape `\"`, x, then the closing quote. `"\\"`: one escape `\\`, then the closing quote.
`"x`: no closing quote. `"\"`: the `\"` must be read as an escape (a body item), so the closing quote is missing. `"x"x"`: after `"x"` the string is complete; the remaining `x"` cannot be part of it because a bare `"` is not a body item.

Gotcha

Writing the body as "anything": `"`$(x \mid \texttt{"} \mid \backslash)^{*}$`"`. That accepts `"x"x"`, and with longest match it swallows two strings plus the code between them into one token.

3 Thompson's construction: regex to NFA [course]

Method

Build the NFA from the inside out; every fragment has one start and one final state.

- Symbol a: two states, $i \xrightarrow{a} f$.
- **Concatenation st, convention B** (the textbook's): the final state of $N(s)$ *becomes* the start state of $N(t)$; no state is added.
- Union $s \mid t$: a new start with ε-edges to both starts; both finals have ε-edges to a new final.
- Star s^*: a new start i and a new final f; ε-edges $i \to$ (start of s), (final of s)$\to$ (start of s) for "again", (final of s)$\to f$, and $i \to f$ for "zero times".

Counting states and edges. Parse the regex first and count, in its tree: n_ℓ leaves (symbols, and ε if it appears), $n_|$ unions, n_* stars, $n_\cdot$ concatenations (`abc` has two). Rewrite $r?$ as $(r \mid \varepsilon)$ and r^+ as rr^* before counting.

	States	Edges (transitions)
Convention A (ε-edge per concatenation)	$2(n_\ell + n_\mid + n_*)$	$n_\ell + 4(n_\mid + n_*) + n_\cdot$
Convention B (concatenation merges two states)	$2(n_\ell + n_\mid + n_*) - n_\cdot$	$n_\ell + 4(n_\mid + n_*)$

Why: a leaf adds 2 states and 1 edge; a union or a star adds 2 states and 4 ε-edges; a concatenation adds one ε-edge (A) or merges two states into one (B). Equivalent form for B: $2(\#\text{operators} + \#\text{operands}) - 3n_\cdot$, with concatenation counted among the operators.

Practice

Given. The regular expression $a(b \mid c)^*$.

Your task. (a) Build its NFA with Thompson's construction, convention B. Number the states and draw it. (b) Count the states and the edges of your drawing, and check both counts with the formulas. (c) Give the numbers of states and edges under convention A (which joins two fragments with an extra ε-edge instead of merging states).

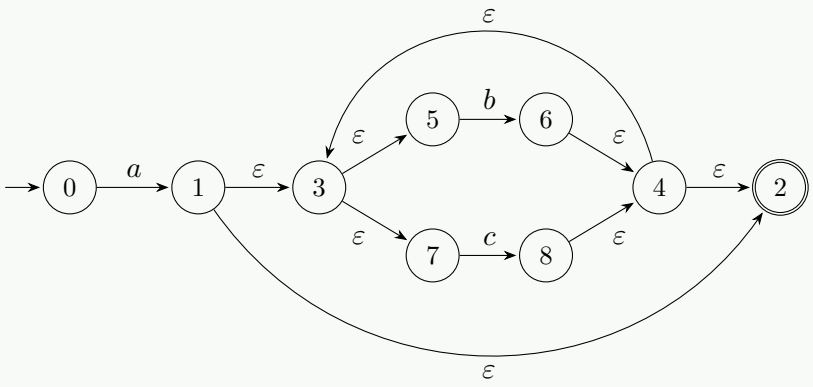
Solution

Step 1, innermost pieces. b: $5 \xrightarrow{b} 6$. c: $7 \xrightarrow{c} 8$.

Step 2, union $b \mid c$. New start 3 and final 4: $3 \xrightarrow{\varepsilon} 5$, $3 \xrightarrow{\varepsilon} 7$, $6 \xrightarrow{\varepsilon} 4$, $8 \xrightarrow{\varepsilon} 4$.

Step 3, star. New start 1 and final 2: $1 \xrightarrow{\varepsilon} 3$, $4 \xrightarrow{\varepsilon} 3$ (again), $4 \xrightarrow{\varepsilon} 2$, $1 \xrightarrow{\varepsilon} 2$ (zero times).

Step 4, concatenation with a. a is $0 \xrightarrow{a} 1$: its final state *is* the star's start 1.



(b) Counts in the tree: $n_\ell = 3$ (a, b, c), $n_| = 1$, $n_* = 1$, $n_. = 1$. States in the drawing: 9 (0–8). Formula B: $2(3+1+1)-1 = 9$. ✓ Edges in the drawing: 11 (three symbol edges, eight ε-edges). Formula B: $3 + 4(1+1) = 11$. ✓

(c) Convention A keeps the final state of $N(a)$ and the start of the star apart and links them by one ε-edge: $2(3+1+1) =$ **10 states** and $3 + 4(1+1) + 1 =$ **12 edges**. (Same counts as Lecture 3's $(0 \mid 1)^*1$: 10 states under A, 9 under B.)

Gotcha

State the convention: the counts differ by exactly $n_.$ states and $n_.$ edges. Count concatenations in the parse tree (`abc` has two), and expand r^+ to rr^* first (its leaves count twice). The star needs *two* extra edges besides the entry and exit: the back edge (repeat) and the bypass $i \to f$ (zero times). Forgetting the bypass builds $(b \mid c)^+$.

4 Simulating an NFA: ε-closure and move [course]

Method

1. Current set $S = \varepsilon$-closure($\{$start$\}$): the start plus everything reachable by ε-edges alone.
2. For each input symbol c: move(S, c) = the states reached by one c-edge from S; then take the ε-closure of that.
3. At the end, accept iff the set contains an accepting state. An empty set means reject at once.

Practice

Given. The NFA for $a(b \mid c)^*$ drawn in the solution of §3 (start 0, accepting 2).

Your task. (a) Simulate the input `acb`: write the set of states after each symbol, and say whether the input is accepted. (b) Do the same for `ca`.

Solution

(a)

Read	move	ε-closure (the new set)	contains 2?
(start)	—	$\{0\}$	no
`a`	$\{1\}$	$\{1, 3, 2, 5, 7\}$	yes
`c`	$\{8\}$	$\{8, 4, 3, 2, 5, 7\}$	yes
`b`	$\{6\}$	$\{6, 4, 3, 2, 5, 7\}$	yes

The final set contains 2: **accepted**.

(b) From $\{0\}$, there is no c-edge: move$(\{0\}, c) = \emptyset$. **Rejected** after the first symbol.

Gotcha

Take the closure after *every* move. Without the closure after `a`, the set would be $\{1\}$ and the following `c` would find no c-edge, wrongly rejecting.

5 Subset construction: NFA to DFA [mock 1.7(a)]

Method

1. DFA start state: $A = \varepsilon\text{-closure}(\{s_0\})$.
2. For each new set S and each symbol c: compute $T = \varepsilon\text{-closure}(\text{move}(S, c))$; give it a name if it is new.
3. Stop when no new set appears. A DFA state is accepting iff its set contains an accepting NFA state.

Practice

Given. An NFA over $\{a, b\}$ with states 0, 1, 2; start 0; accepting state 2; no ε-edges. Edges: $0 \xrightarrow{a} 0$, $0 \xrightarrow{b} 0$, $0 \xrightarrow{a} 1$, $1 \xrightarrow{a} 2$, $1 \xrightarrow{b} 2$. It accepts the strings whose second symbol from the end is a.

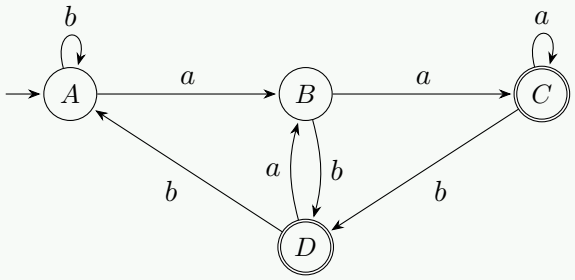
Your task. (a) Build the DFA with the subset construction: give the table (set, on a, on b, accepting?). (b) Draw it. (c) Explain why no DFA for this language has fewer states.

Solution

(a) No ε-edges, so every closure is the set itself.

Name	Set	on a	on b	Accepting?
A	$\{0\}$	$\{0,1\} = B$	$\{0\} = A$	no
B	$\{0,1\}$	$\{0,1,2\} = C$	$\{0,2\} = D$	no
C	$\{0,1,2\}$	$\{0,1,2\} = C$	$\{0,2\} = D$	yes (contains 2)
D	$\{0,2\}$	$\{0,1\} = B$	$\{0\} = A$	yes

(b)



(c) The DFA must remember the last two symbols: $bb \to A$, $ba \to B$, $aa \to C$, $ab \to D$. These four strings are pairwise distinguishable (§10), so 4 states is the minimum.

Gotcha

A DFA state is accepting if its set contains *at least one* accepting NFA state. And the set is unordered: $\{0, 1\}$ and $\{1, 0\}$ are the same state; naming them twice creates a fake extra state.

6 DFA minimisation by table filling [course]

Method

1. List all pairs of states (add a dead state if some transitions are missing).
2. Mark every pair made of one accepting and one non-accepting state.
3. Sweep: mark a pair (p, q) if some symbol sends it to a pair already marked. Repeat full sweeps until one sweep marks nothing.
4. Unmarked pairs are equivalent: merge them, and redirect every edge that pointed at a merged state.

Practice

Given. A DFA over $\{a, b\}$ (it accepts $(a \mid b)^*ab$), start A, accepting state C:

State	on a	on b
A (start)	B	D
B	B	C
C (accepting)	B	D
D	B	D

Your task. (a) Fill the table of pairs: for each pair, say in which step or sweep it is marked, or that it is never marked. (b) Draw the minimal DFA.

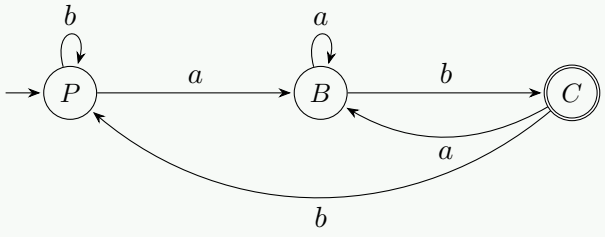
Solution

(a) 4 states, so 6 pairs.

Pair	Marked when	Why
(A, C), (B, C), (D, C)	step 2	C accepts, the other does not
(A, B)	sweep 1	on b: (D, C), already marked
(B, D)	sweep 1	on b: (C, D), already marked
(A, D)	never	on a: (B, B); on b: (D, D), same states

Sweep 2 marks nothing, so stop. $A \equiv D$.

(b) Merge A and D into P:



Meaning: P "nothing useful", B "just read a", C "just read ab".

Gotcha

After merging, redirect *every* edge into D (here $A \xrightarrow{b} D$, $C \xrightarrow{b} D$, $D \xrightarrow{b} D$) to P. And do one last sweep that marks nothing before you stop.

7 State elimination: DFA to regex [course]

Method

1. If the start state has incoming edges, add a new start s with $s \xrightarrow{\varepsilon}$(old start). Add a new final f with ε-edges from every accepting state.
2. Remove the other states one by one. To remove q with self-loop L: for every path $p \xrightarrow{X} q \xrightarrow{Y} r$, add the edge $p \xrightarrow{XL^*Y} r$ (if $p = r$ this is a self-loop), joined by $\mid$ with any existing $p \to r$ edge.
3. When only s and f remain, the label of $s \to f$ is the answer.

Practice

Given. A DFA over $\{a, b\}$ with states 0 (start and accepting) and 1: $0 \xrightarrow{b} 0$, $0 \xrightarrow{a} 1$, $1 \xrightarrow{b} 1$, $1 \xrightarrow{a} 0$.

Your task. (a) Find a regular expression for its language by state elimination, removing state 1 first. (b) Describe the language in words.

Solution

Step 1. State 0 has incoming edges, so add $s \xrightarrow{\varepsilon} 0$; add $0 \xrightarrow{\varepsilon} f$.

Step 2, remove 1. Its loop is b. The only path through 1 is $0 \xrightarrow{a} 1 \xrightarrow{a} 0$, which becomes the loop ab^*a on 0. With the existing loop b, state 0 now has the loop $b \mid ab^*a$.

Step 3, remove 0. Path $s \xrightarrow{\varepsilon} 0 \xrightarrow{\varepsilon} f$ through the loop: $s \xrightarrow{(b|ab^*a)^*} f$.

(a) $(b \mid ab^*a)^*$ **(b)** strings with an even number of a's.

Gotcha

Without the new start state you would read 0's loop and forget that the expression must also allow leaving and re-entering 0. When the start has incoming edges, always add s.

8 Brzozowski derivatives [course]

Method

$\partial_a R$ is "what remains of R after reading a". $\nu(R) = \varepsilon$ if R accepts the empty string, otherwise $\emptyset$.

$$\partial_a a = \varepsilon, \quad \partial_a b = \emptyset, \quad \partial_a \varepsilon = \partial_a \emptyset = \emptyset, \quad \partial_a(R \mid S) = \partial_a R \mid \partial_a S, \quad \partial_a(R^*) = (\partial_a R)\, R^*,$$
$$\partial_a(RS) = (\partial_a R)\, S \;\mid\; \nu(R)\, \partial_a S.$$

Read w by taking derivatives symbol by symbol; accept iff the final expression is nullable. The distinct expressions are the states of a DFA.

Practice

Given. $R = (ab)^*$ over $\{a, b\}$.

Your task. (a) Compute $\partial_a R$ and $\partial_b R$, then the derivatives of every new expression, until no new expression appears. (b) Decide `abab` and `aba` with these derivatives. (c) Draw the resulting DFA.

Solution

(a) $\partial_a(ab)^* = (\partial_a(ab))(ab)^*$. Since a is not nullable, $\partial_a(ab) = (\partial_a a)b = \varepsilon b = b$. So $\partial_a R = b(ab)^* =: R_1$. $\partial_b R = (\partial_b(ab))(ab)^* = \emptyset$. Next, $R_1 = b(ab)^*$: $\partial_a R_1 = \emptyset$ ($\partial_a b = \emptyset$, and b is not nullable); $\partial_b R_1 = \varepsilon(ab)^* = R$. No new expression: the states are R, R_1, $\emptyset$.

(b) `abab`: $R \to R_1 \to R \to R_1 \to R$; $\nu(R) = \varepsilon$: **accepted**. `aba`: ends in R_1, not nullable: **rejected**.

(c) R (start, accepting) $\xrightarrow{a} R_1$, $R_1 \xrightarrow{b} R$; every other move goes to the dead state $\emptyset$.

Gotcha

In $\partial_a(RS)$ the second term $\nu(R)\, \partial_a S$ is needed when R can be empty (e.g. $R = x^*$): the a may then belong to S. Leaving it out rejects strings like `a` for $x^* a$.

9 Designing a DFA directly from a description [mock 2.3(b)]

Method

1. Decide what the DFA must *remember* about what it has read so far (a remainder, how much of a pattern has been seen, inside or outside a comment).
2. One state per possible value of that memory; the start state is the value for the empty string.
3. For each state and symbol, compute the new memory value. Mark the accepting values.
4. Test on two strings that must be accepted and two that must be rejected.

Practice

Given. Binary strings over $\{0, 1\}$, read from the most significant bit (e.g. `110` is 6).

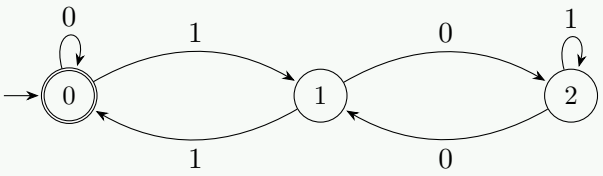
Your task. (a) Build a DFA that accepts exactly the binary numbers divisible by 3 (the empty string counts as 0). (b) Run it on `110` and `111`.

Solution

Step 1. Remember the remainder r of the number read so far, divided by 3.

Step 2. States $0, 1, 2$; start 0 (the empty string is 0).

Step 3. Reading bit b turns the number n into $2n + b$, so $r \mapsto (2r + b) \bmod 3$. Accepting: 0.



(b) `110`: $0 \xrightarrow{1} 1 \xrightarrow{1} 0 \xrightarrow{0} 0$: accepted ($6 = 3 \times 2$). `111`: $0 \to 1 \to 0 \to 1$: rejected (7).

Gotcha

Reading a bit *doubles* the number first: the update is $(2r + b) \bmod 3$, not $(r + b) \bmod 3$.

10 Proving a language is not regular; DFA lower bounds [mock 1.2, 2.3(c)]

Method

Two strings u, v are **distinguishable** if one suffix z puts exactly one of uz, vz in L. A DFA must send them to different states.

- Infinitely many pairwise distinguishable strings $\Rightarrow$ L is not regular.
- k pairwise distinguishable strings $\Rightarrow$ every DFA for L has at least k states.

Template: "take $u = a^i$, $v = a^j$ with $i < j$; the suffix $z = \ldots$ gives $uz \in L$ and $vz \notin L$."

Practice

Given. Three languages over $\{a, b\}$.

Your task. (a) Prove that $L_1 = \{a^n b^m : m \geq n \geq 0\}$ is not regular. (b) Is $L_2 = \{a^n b^m : n, m \geq 0\}$ regular? Justify. (c) L_3 = strings whose third symbol from the end is a. Prove that every DFA for L_3 has at least 8 states.

Solution

(a) Take $u = a^i$, $v = a^j$ with $i < j$ and $z = b^i$. Then $uz = a^i b^i \in L_1$ ($i \geq i$) and $vz = a^j b^i \notin L_1$ ($i < j$). So all $a^0, a^1, a^2, \ldots$ are pairwise distinguishable: infinitely many states would be needed; L_1 is not regular.

(b) Yes: $L_2 = a^* b^*$, a regular expression. n and m are independent; nothing must be counted.

(c) Take the 8 strings of length 3. If $u \neq v$, let k be the first position (from the left) where they differ. Append $z = b^{k-1}$: position k becomes the third from the end, a in one string and b in the other, so exactly one is in L_3. 8 pairwise distinguishable strings: at least 8 states.

Gotcha

Exponents do not mean non-regular. Non-regularity needs a *link* between counts (as $m \geq n$) that the automaton would have to count without bound.

11 The combined scanner DFA and rule priority [mock 2.2, 2.4]

Method

1. Choose character classes (characters that behave the same in every rule).
2. Build one DFA for all rules together (by hand, or NFA per rule + subset construction).
3. Label each accepting state with the **first-listed** rule that accepts there.
4. A rule that labels no state can never fire.

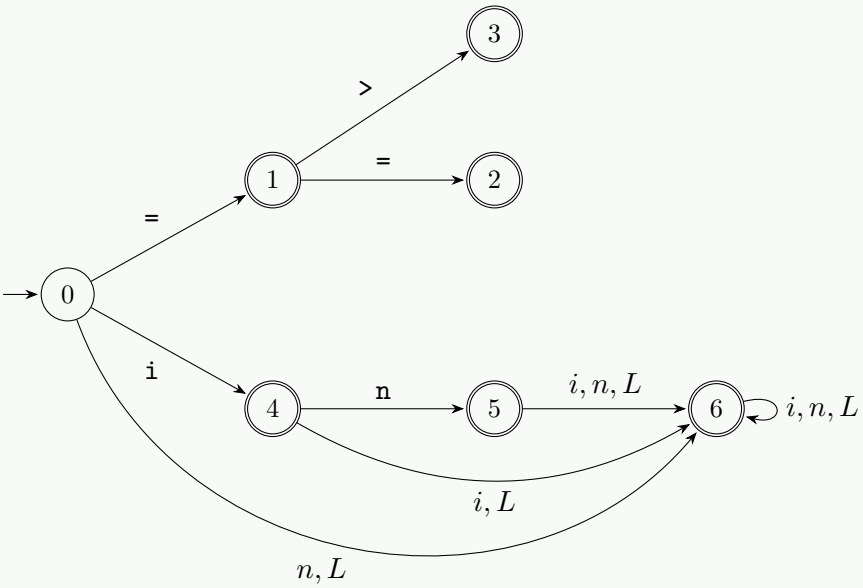
Practice

Given. Lexer rules, in this order: (1) `=` $\rightarrow$ ASSIGN; (2) `==` $\rightarrow$ EQ; (3) `=>` $\rightarrow$ ARROW; (4) `[a-z]+` $\rightarrow$ ID; (5) `in` $\rightarrow$ IN. Character classes: `=`, `>`, `i`, `n`, and L for every other lowercase letter.

Your task. (a) Draw the combined DFA and label each accepting state with its token. (b) Which rule can never fire, and why? (c) Give the token sequence for the input `in=>inx==`. (d) Fix the specification so that `in` is a keyword.

Solution

(a)



1 ASSIGN, 2 EQ, 3 ARROW, 4 ID, 6 ID; state 5 (reached by `in`) accepts for both rule 4 and rule 5, and is labelled ID because rule 4 is listed first.

(b) Rule 5 (IN) labels no state: it can never fire.

(c) ID(in), ARROW, ID(inx), EQ.

(d) Move `in` above the identifier rule. State 5 becomes IN; `inx` still reaches state 6 and stays ID.

Gotcha

Priority decides only *ties in length*. Even with the keyword first, `inx` is an identifier, because the identifier match is longer.

12 Longest-match scanning with backup, counting reads [mock 2.1]

Method

1. From the start state, keep reading while a transition exists. Each time you are in an accepting state, remember it and the position.
2. When stuck (or at end of input), back up to the last remembered position, emit its token, and restart from there. The characters read beyond it are read again later (pushback).
3. If no accepting state was seen: lexical error on one character, skip it.

Practice

Given. Rules, in this order: (1) `a`; (2) `abb`; (3) `a*b+`.

Your task. (a) Scan `abbbaab`: list the tokens with the rule that produced each, the number of characters pushed back, and the total number of characters read. (b) With only the rules `a` and `a*b`, how many characters are read in total on the input `aaaa`?

Solution

(a) First token, from position 0:

Read so far	Rules that match	Remembered	Continue?
`a`	1 (and 3 could continue)	rule 1, length 1	yes
`ab`	3	rule 3, length 2	yes
`abb`	2 and 3 (tie: rule 2)	rule 2, length 3	yes
`abbb`	3	rule 3, length 4	yes
`abbba`	none, stuck	—	back up 1

Token **rule 3,** `abbb`, pushback 1, 5 characters read. Second token, from `aab`: `a` (rule 1), `aa` (none yet, but `a*b+` can continue), `aab` (rule 3), end of input: token **rule 3,** `aab`, 3 characters read, pushback 0. Total: $5 + 3 = \mathbf{8}$ **reads**.

(b) Every token reads to the end hoping for a `b`, then backs up to one `a`: $4 + 3 + 2 + 1 = \mathbf{10}$ **reads** for four tokens `a`.

Gotcha

Do not stop at the first accepting state: `abb` matches rule 2, yet the scanner continues and returns `abbb` by rule 3. And when stuck in a non-accepting state, back up to the last *accepting* one.

13 Lexer states for nested comments [mock 2.3(c)]

Method

When a token is not regular (nested comments), switch the lexer into a **state** (flex: start condition) with its own rules, and keep the extra memory in a variable (a depth counter). Return to the normal state when the construct closes, and add a rule for end of file inside it.

Practice

Given. Comments `/* ... */` that may be nested.

Your task. (a) Write flex-style rules with a start condition `COMMENT` and a counter `depth`. (b) Give the value of `depth` after each `/*` and `*/` in `/* a /* b */ c */ x`, and the token returned after the comment.

Solution

(a)

```
"/*"             { depth = 1; BEGIN(COMMENT); }
<COMMENT>"/*"    { depth++; }
<COMMENT>"*/"    { if (--depth == 0) BEGIN(INITIAL); }
<COMMENT>.|\n    { /* skip */ }
<COMMENT><<EOF>> { error("unterminated comment"); }
```

(b) First `/*`: 1. Second `/*`: 2. First `*/`: 1. Second `*/`: 0, back to `INITIAL`. Then `x` is returned as ID.

Gotcha

Without the `«EOF»` rule, an unclosed comment silently swallows the rest of the file instead of being a lexical error.

Part B. Grammars and top-down parsing

14 Writing a context-free grammar for a language [mock 1.2(b)]

Method

The idea. A rule like $S \to a\,S\,b$ adds one a on the left *and* one b on the right in the same step. That is the only way a grammar can keep two counts equal: produce the two symbols *together*.

1. **Write the strings as blocks.** Split the language into blocks of one letter each, and split sums: $a^n b^m c^{n+m}$ is $a^n\ b^m\ c^m\ c^n$.
2. **Link the blocks whose counts must be equal**, and draw the links as arcs over the blocks: $a^n \dots c^n$ and $b^m\,c^m$. The arcs must *nest* (one inside the other, or side by side), like parentheses. If two arcs cross, as in $a^n b^m c^n d^m$, no CFG exists.



3. **One nonterminal per arc, from the outside in.** The outermost arc gets a *wrap* rule that adds one symbol on each side and recurses in the middle, plus an *exit* to the next layer: $S \to a\,S\,c \mid B$. The next arc inside: $B \to b\,B\,c \mid \varepsilon$.
4. **Blocks with no link** get a free-repetition rule: $A \to a\,A \mid \varepsilon$. Blocks side by side are concatenated: $S \to X\,Y$.
5. **Check**: derive the smallest strings, and for each rule say what it adds, so that every derived string has the right counts.

Toolbox (one line each):

Language	Grammar
$a^n b^n$ (equal, nested)	$S \to aSb \mid \varepsilon$
$a^n b^{2n}$ (fixed ratio)	$S \to aSbb \mid \varepsilon$
$a^n b^n c^m$ (one arc, then a free block)	$S \to XC$, $X \to aXb \mid \varepsilon$, $C \to cC \mid \varepsilon$
$a^n b^m c^m d^n$ (arc inside an arc)	$S \to aSd \mid B$, $B \to bBc \mid \varepsilon$
$L_1 \cup L_2$ (two cases)	$S \to S_1 \mid S_2$, one grammar per case
equal numbers of a and b, any order	$S \to aSbS \mid bSaS \mid \varepsilon$

Practice

Given. Three languages.

Your task. Write a context-free grammar for each: (a) $\{a^n b^m c^{n+m} : n, m \geq 0\}$ (e.g. `abcc`, `bc`, `aacc`); (b) $\{a^i b^j : i \leq j \leq 2i\}$ (each a is followed, later, by one or two b's; e.g. `ab`, `abb`, `aabbb`); (c) all strings over $\{a, b\}$ with as many a's as b's.

Solution

(a) Step 1, blocks: $a^n b^m c^{n+m} = a^n\ b^m\ c^m\ c^n$. Step 2, arcs: a^n with the last c^n (outer), b^m with c^m (inner); they nest. Step 3, outer arc $S \to a\,S\,c \mid B$, inner arc $B \to b\,B\,c \mid \varepsilon$:

$$S \to a\,S\,c \mid B, \qquad B \to b\,B\,c \mid \varepsilon.$$

Step 5, check: $S \Rightarrow aSc \Rightarrow aBc \Rightarrow abBcc \Rightarrow abcc$ ($n = m = 1$); $S \Rightarrow B \Rightarrow bBc \Rightarrow bc$ ($n = 0$). Each S-step adds one a and one c, each B-step one b and one c: $\#c = n + m$.

(b) One arc, a^i with b^j, but the ratio may vary between 1 and 2: give the wrap rule two versions, one adding one b, one adding two:

$$S \to a\,S\,b \mid a\,S\,b\,b \mid \varepsilon.$$

With i steps of which k add two b's: $j = i + k$ with $0 \le k \le i$, exactly $i \le j \le 2i$.

(c) The toolbox's last line. Reason: a nonempty balanced string starts with a or b; cut it at the symbol where the counts first become equal again; the part inside and the part after are balanced:

$$S \to a\,S\,b\,S \mid b\,S\,a\,S \mid \varepsilon.$$

Gotcha

$S \to aS \mid Sb \mid \varepsilon$ looks right for "a's then b's with equal counts" but generates a^*b^*: the a and the b are added in *different* steps, so nothing links their numbers.

15 Leftmost and rightmost derivations, parse trees [mock 1.3]

Method

The idea. A derivation is a list of strings (sentential forms). Each step replaces *one* nonterminal by the right side of *one* of its productions. "Leftmost" and "rightmost" only say *which* nonterminal you replace when there are several: always the leftmost one, or always the rightmost.

1. **Build the parse tree first.** Start at S. Choose the production of the root that can produce the sentence (look at its first and last symbols and at the length), then do the same for each child until every leaf is a terminal. A tree is easier to get right than a list of strings.
2. **Leftmost derivation = walk the tree depth-first, left child first.** Each time you meet an interior node, write one step: copy the previous string and replace that node's nonterminal (it is always the leftmost one still in the string) by its children.
3. **Rightmost derivation = the same walk, right child first.** Each step replaces the rightmost nonterminal.
4. **Check**: both derivations have exactly as many steps as the tree has interior nodes, start at S and end at the sentence. Write the whole string at every step; never skip one.

Mini-example. $S \to AB$, $A \to a$, $B \to b$, sentence `ab`. Tree: $S(A(a), B(b))$, 3 interior nodes. Leftmost: $S \underset{lm}{\Rightarrow} AB \underset{lm}{\Rightarrow} aB \underset{lm}{\Rightarrow} ab$. Rightmost: $S \underset{rm}{\Rightarrow} AB \underset{rm}{\Rightarrow} Ab \underset{rm}{\Rightarrow} ab$. Same tree, same 3 steps, different middle string.

Practice

Given. The postfix-expression grammar $S \to S\,S\,\texttt{+} \mid S\,S\,\texttt{*} \mid a$ and the sentence `aa+a*`.

Your task. (a) Give a leftmost derivation. (b) Give a rightmost derivation. (c) Draw the parse tree.

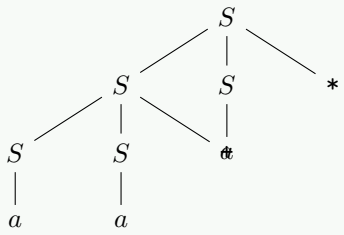
Solution

Step 1. The last symbol is `*`, so the root uses $S \to SS*$: the first S gives `aa+`, the second gives `a`. `aa+` uses $S \to SS+$ with two a's.

(a) $S \underset{lm}{\Rightarrow} SS* \underset{lm}{\Rightarrow} SS+S* \underset{lm}{\Rightarrow} aS+S* \underset{lm}{\Rightarrow} aa+S* \underset{lm}{\Rightarrow} aa+a*$

(b) $S \underset{rm}{\Rightarrow} SS* \underset{rm}{\Rightarrow} Sa* \underset{rm}{\Rightarrow} SS+a* \underset{rm}{\Rightarrow} Sa+a* \underset{rm}{\Rightarrow} aa+a*$

(c)



Gotcha

In the rightmost derivation the first steps build the *right* end of the sentence (here the final `a*`), so the intermediate strings look very different from the leftmost ones. Both end in the same tree.

16 Ambiguity, precedence and associativity [mock 1.4(a)–(c)]

Method

The idea. A grammar is ambiguous if *one* sentence has *two* parse trees. For expression grammars the two trees are always two ways of grouping the operators: $(a+b)*c$ or $a+(b*c)$ (precedence), $(a-b)-c$ or $a-(b-c)$ (associativity).

To prove ambiguity:

1. Pick the shortest sentence with two operators, e.g. id + id * id.
2. Draw the tree that groups the *left* pair first, then the tree that groups the *right* pair first.
3. Say what each tree computes. Two trees, one sentence: ambiguous.

To remove it (layering):

1. List the operators from *loosest* to *tightest* binding, e.g. $+$, then $*$, then ^.
2. Make one nonterminal per level, plus one for atoms: E $(+)$, T $(*)$, F $(\hat{})$, P (atoms). Each level is built from the next one down.
3. For each level write one rule, chosen by associativity (X = this level, Y = next level):

Associativity	Rule	Tree leans
left $(a-b-c=(a-b)-c)$	$X \to X\ op\ Y \mid Y$	to the left
right $(a\hat{}b\hat{}c=a\hat{}(b\hat{}c))$	$X \to Y\ op\ X \mid Y$	to the right
none ($a<b<c$ is an error)	$X \to Y\ op\ Y \mid Y$	—
prefix unary minus	$X \to -\,X \mid Y$	—

4. Atoms: $P \to (\ E\ ) \mid$ id. The parentheses go back to the *top* level E, so anything can be put inside them.
5. **Check**: draw the tree of a mixed sentence and verify that the tightest operator sits lowest.

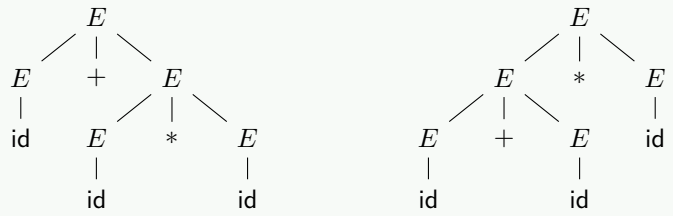
Practice

Given. $E \to E+E \mid E*E \mid E\ \hat{}\ E \mid (E) \mid$ id.

Your task. (a) Show that it is ambiguous using id + id * id. (b) Write an unambiguous grammar for the same language in which ^ binds tightest and is right-associative, * is next, + binds loosest, and +, * are left-associative. (c) Draw the parse tree of id ^ id ^ id in your grammar.

Solution

(a) Two trees for the same sentence:

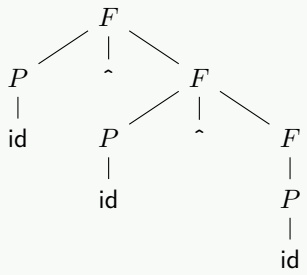


The left tree computes $\mathsf{id} + (\mathsf{id} * \mathsf{id})$, the right one $(\mathsf{id} + \mathsf{id}) * \mathsf{id}$.

(b) Four levels, loosest first:

$$E \to E + T \mid T, \qquad T \to T * F \mid F, \qquad F \to P \mathbin{\hat{}} F \mid P, \qquad P \to (E) \mid \mathsf{id}.$$

(c) $F \to P \mathbin{\hat{}} F$ is right-recursive, so the tree nests to the right:



(Above F the tree continues $E \to T \to F$.) It means $\mathsf{id}^{(\mathsf{id}^{\mathsf{id}})}$.

Gotcha

A leftmost and a rightmost derivation of the same sentence do not prove ambiguity: every unambiguous sentence has both. You need two different *trees* (equivalently, two different *leftmost* derivations).

17 Recursive descent: grammar to code and back [mock 4.1, 4.2(b), 4.5]

Method

The idea. One procedure per nonterminal. The procedure for A must parse "something A derives". It looks at the current token (the *lookahead*, `la`) to choose which production of A to follow, then follows its right side from left to right.

1. **Selection sets.** For each production $A \to \alpha$: SEL = FIRST(α) \ $\{\varepsilon\}$, plus FOLLOW(A) if α can vanish (§19–21).
2. **Write each procedure from this template:**

```
void A() {
    if      (la in SEL(A -> alpha1)) { /* body of alpha1 */ }
    else if (la in SEL(A -> alpha2)) { /* body of alpha2 */ }
    else error();
}
void match(Token t) { if (la == t) la = nextToken(); else error(); }
```

3. **The body of a right side**: for each symbol from left to right, a terminal t becomes `match(t)`, a nonterminal B becomes `B()`. An ε body is empty.
4. **Code to grammar (reverse)**: each `if` branch is one production; the `match`/call sequence in it is the right side; the condition is its selection set. A branch that does nothing is an ε-production.
5. **Call trace**: start with the start symbol's procedure. Write every call and every `match` in the order they happen, indented by depth. At each procedure, note which token decided the branch. A `match` consumes one token, a call consumes nothing by itself.

Practice

Given. The list grammar $S \to (\ L\ ) \mid a, \quad L \to S\,L', \quad L' \to ,\ S\,L' \mid \varepsilon$. (Use `la` for the lookahead token and `Lp` for L'.)

Your task. (a) Write the procedures `S()`, `L()` and `Lp()`. (b) List, in order, every call to a procedure or to `match` made while parsing `(a,a)`.

Solution

Selection sets. $S \to (L)$: {(}; $S \to a$: {a}; $L \to SL'$: FIRST$(S) = \{(, a\}$; $L' \to , SL'$: {,}; $L' \to \varepsilon$: FOLLOW$(L') = \{)\}$.

(a)

```
void S()  { if (la == '(')      { match('('); L(); match(')'); }
            else if (la == 'a')   match('a');
            else error(); }
void L()  { if (la == '(' || la == 'a') { S(); Lp(); }
            else error(); }
void Lp() { if (la == ',')      { match(','); S(); Lp(); }
            else if (la == ')')   { /* eps */ }
            else error(); }
```

(b) Indented by depth, with the lookahead that decided each branch:

```
S()           la = '('  -> S -> ( L )
  match('(')
  L()         la = 'a'  -> L -> S L'
    S()       la = 'a'  -> S -> a
      match('a')
    Lp()      la = ','  -> L' -> , S L'
      match(',')
      S()     la = 'a'  -> S -> a
        match('a')
      Lp()    la = ')'  -> L' -> eps  (returns)
  match(')')
```

Practice: the reverse: code to grammar

Given. A recursive-descent parser for variable declarations. `INT`, `BOOL`, `ID`, `NUM` are token kinds; quoted characters are their own tokens.

```
void D()  { if (la == INT || la == BOOL) { Ty(); match(ID); R(); match(';'); }
            else error(); }
void Ty() { if (la == INT)       match(INT);
            else if (la == BOOL) match(BOOL);
            else error(); }
void R()  { if (la == '[')       { match('['); match(NUM); match(']'); }
            else if (la == '=')  { match('='); Ex(); }
            else if (la == ';')  { /* nothing */ }
            else error(); }
void Ex() { if (la == ID || la == NUM) { A(); while (la == '+') { match('+'); A(); } }
            else error(); }
void A()  { if (la == ID)        match(ID);
            else if (la == NUM)  match(NUM);
            else error(); }
```

Your task. (a) Write the grammar this code parses, numbered, with the start symbol. (b) Give the selection set of every production and say where each one is visible in the code. (c) Which nonterminals are nullable, and what does the code tell you about their FOLLOW sets? (d) Is the grammar LL(1)? Answer from the code alone.

Solution

(a) One procedure = one nonterminal, one `if` branch = one production. The right side is the sequence of `match` calls (terminals) and procedure calls (nonterminals) inside the branch, in order. The `while` loop in `Ex()` is EBNF iteration $\{ + A \}$: it stands for a nonterminal of its own, right-recursive, with an ε-production for the exit. Start symbol: the procedure nothing else calls, D.

1	$D \to Ty$ `ID` R `;`	7	$Ex \to A\, Ex'$
2	$Ty \to$ `INT`	8	$Ex' \to$ `+` $A\ Ex'$
3	$Ty \to$ `BOOL`	9	$Ex' \to \varepsilon$
4	$R \to$ `[` `NUM` `]`	10	$A \to$ `ID`
5	$R \to$ `=` Ex	11	$A \to$ `NUM`
6	$R \to \varepsilon$		

(b) Selection set = the `if` condition. 1: {`INT`,`BOOL`} (this is FIRST(Ty), the first symbol of the right side is a nonterminal, so the condition is *its* FIRST). 2: {`INT`}. 3: {`BOOL`}. 4: {`[`}. 5: {`=`}. 6: {`;`}, the empty branch. 7: {`ID`,`NUM`} = FIRST(A). 8: {`+`}, the loop condition. 9: the loop exit, "anything but `+`"; the code never tests it, the grammar says FOLLOW(Ex'). 10: {`ID`}. 11: {`NUM`}.

(c) Nullable: R and Ex', the two with an empty branch or a loop exit. Their FOLLOW sets are read from the empty branches and from what the *caller* does next: `R()` takes its empty branch on `;` and `D()` matches `;` right after `R()`, so FOLLOW(R) = {`;`}. Ex' ends Ex, Ex ends R (production 5), so FOLLOW(Ex') = FOLLOW(Ex) = FOLLOW(R) = {`;`}: the loop stops on `;` and `D()` consumes it. (A stray token such as `)` after `a+3` also leaves the loop; it is then caught by `match(';')` in `D()`, so the parser still reports an error, one call later.)

(d) LL(1): yes. In every procedure the branch conditions are pairwise disjoint (`Ty`: `INT`/`BOOL`; `R`: `[`/`=`/`;`; `A`: `ID`/`NUM`; the loop: `+` versus FOLLOW(Ex') = {`;`}), and no procedure calls itself first. Disjoint selection sets for every nonterminal is exactly the LL(1) condition, so the code's existence is the proof.

Gotcha

A left-recursive production $A \to A\alpha$ becomes `A(){ A(); ... }`: `A` calls itself with the same lookahead, consumes nothing, and overflows the stack. Remove left recursion first (§24).

In the reverse direction, two traps. (1) A token in an `if` condition belongs to the right side *only if a `match` of it follows*: `if (la == '(') { match('('); ...}` puts `(` in the production; `if (la == ';') { }` does not, `;` is a FOLLOW token that the caller consumes. (2) A `while (la == t) {...}` loop is not a production of the enclosing nonterminal: it is a new right-recursive nonterminal $X' \to t \ldots X' \mid \varepsilon$. Forget it and the grammar accepts only one operand.

18 Nullable nonterminals [mock 1.6(a), 5(a)]

Method

The idea. A is *nullable* if A can derive the empty string ε. It matters because a nullable symbol can disappear, so the parser must look past it (FIRST) or after the nonterminal (FOLLOW).

1. **Pass 1**: mark every A that has a production $A \to \varepsilon$.
2. **Each further pass**: go through the productions in the order written. Mark A if one of its right sides consists *only of nonterminals that are already marked* (marks made earlier in the same pass count). A right side containing a terminal never qualifies.
3. **Stop** after a pass that marks nothing new.
4. A *string* is nullable iff every symbol in it is nullable; ε itself is nullable.

Write the result as a table: pass, newly marked, the production that justifies each mark.

Mini-example. $S \to AB$, $A \to \varepsilon$, $B \to A \mid b$. Pass 1: A. Pass 2: $S \to AB$ fails (B not yet marked), then $B \to A$ marks B. Pass 3: $S \to AB$ marks S. Pass 4: nothing new, stop.

Practice

Given. $E \to A\,e \mid \varepsilon$, $\quad A \to B\,C$, $\quad B \to \varepsilon \mid b$, $\quad C \to D\,D$, $\quad D \to d \mid \varepsilon$.

Your task. List the nullable nonterminals. Go through the productions in the order written, and say in which pass each one is marked.

Solution

Pass	Newly marked	Reason
1	E, B, D	each has an ε-production ($A \to BC$ and $C \to DD$ are checked before B, D are marked)
2	C	$C \to DD$, D marked
3	A	$A \to BC$, both marked
4	—	nothing new: stop

All five are nullable. E is nullable because of $E \to \varepsilon$, not because of $E \to Ae$ (it contains the terminal e).

Gotcha

One pass is not enough: nullability travels up chains ($D \to C \to A$). And a right side that contains a terminal can never vanish.

19 FIRST sets

[mock 1.5(a), 3.2, 5(a)]

Method

1. FIRST$(t) = \{t\}$ for a terminal; $X \to \varepsilon$ puts ε in FIRST(X).
2. For $X \to Y_1 Y_2 \cdots Y_k$: add FIRST$(Y_1) \setminus \{\varepsilon\}$. If Y_1 is nullable, also add FIRST$(Y_2) \setminus \{\varepsilon\}$, and so on. **Stop at the first non-nullable Y_i.** If all are nullable, add ε.
3. Repeat until no set changes. Easiest order: from the nonterminals that depend on nothing upward.

Practice

Given. $S \to A\,B\,C$, $\quad A \to a\,A \mid \varepsilon$, $\quad B \to b\,B \mid C$, $\quad C \to c \mid \varepsilon$. Start symbol S.

Your task. (a) Find the nullable nonterminals. (b) Compute FIRST of C, B, A, S, showing where each element comes from.

Solution

(a) C ($C \to \varepsilon$), A ($A \to \varepsilon$), then B ($B \to C$), then S ($S \to ABC$): all four.

(b)

Set	Where it comes from	Result
FIRST(C)	$C \to c$ gives c; $C \to \varepsilon$ gives ε	$\{c, \varepsilon\}$
FIRST(A)	$A \to aA$ gives a; $A \to \varepsilon$ gives ε	$\{a, \varepsilon\}$
FIRST(B)	$B \to bB$ gives b; $B \to C$ gives FIRST(C), including ε	$\{b, c, \varepsilon\}$
FIRST(S)	scan ABC: $\{a\}$, A nullable so $+\{b, c\}$, B nullable so $+\{c\}$; all nullable so $+\varepsilon$	$\{a, b, c, \varepsilon\}$

Gotcha

ε goes into FIRST(X) only if *every* symbol of some right side can vanish. In $T \to FT'$ with F not nullable, $\varepsilon \notin$ FIRST(T) even though T' is nullable.

20 FOLLOW sets

[mock 1.5(a), 3.2, 5(a)]

Method

1. Put \$ in FOLLOW(S) for the start symbol.
2. Look at every place A *occurs* on a right side, $B \to \alpha\, A\, \beta$: add FIRST(β) $\setminus \{\varepsilon\}$ to FOLLOW(A).
3. If β is nullable, or empty (A is last): also add FOLLOW(B).
4. Repeat until no set changes. FOLLOW never contains ε.

Practice

Given. The grammar of §19: $S \to A\,B\,C$, $A \to a\,A \mid \varepsilon$, $B \to b\,B \mid C$, $C \to c \mid \varepsilon$, with the FIRST sets found there.

Your task. Compute FOLLOW of S, A, B, C. For each element, name the occurrence it comes from.

Solution

Set	Occurrences and what they add	Result
FOLLOW(S)	start symbol: \$ (S occurs on no right side)	{\$}
FOLLOW(A)	in $S \to A\,BC$: FIRST(BC) $\setminus\, \varepsilon = \{b, c\}$; BC nullable, so +FOLLOW(S). In $A \to aA$: A is last, adds FOLLOW(A) itself (nothing new)	$\{b, c, \$\}$
FOLLOW(B)	in $S \to AB\,C$: FIRST(C) $\setminus\, \varepsilon = \{c\}$; C nullable, so +FOLLOW(S). In $B \to bB$: itself	$\{c, \$\}$
FOLLOW(C)	in $S \to ABC$: last, so FOLLOW(S) = {\$}. In $B \to C$: last, so FOLLOW(B) = $\{c, \$\}$	$\{c, \$\}$

Gotcha

FOLLOW is computed from where A *occurs*, not from A's own productions. The most common miss is an occurrence on the right side of a *different* nonterminal, like C inside $B \to C$.

21 Building the LL(1) table and finding conflicts [mock 1.5(b), 3.3, 5(a)]

Method

1. Number the productions top to bottom.
2. For $A \to \alpha$: put its number in $M[A, a]$ for every terminal $a \in \text{FIRST}(\alpha)$.
3. If α is nullable: also in $M[A, b]$ for every $b \in \text{FOLLOW}(A)$ (\$ included).
4. A cell with two numbers is a conflict. FIRST/FIRST: two alternatives can start with the same token. FIRST/FOLLOW: a nullable alternative's FOLLOW meets another alternative's FIRST.

The tokens under which $A \to \alpha$ appears form its **selection set**.

Practice

Given. The grammar of §19–20 with its FIRST and FOLLOW sets. Number the productions: 1 $S \to ABC$, 2 $A \to aA$, 3 $A \to \varepsilon$, 4 $B \to bB$, 5 $B \to C$, 6 $C \to c$, 7 $C \to \varepsilon$.

Your task. (a) Fill the LL(1) table (rows S, A, B, C; columns $a, b, c, \$$). (b) Is the grammar LL(1)? If not, give the cell, the two productions, the kind of conflict, and a sentence that shows why the parser cannot choose.

Solution

Selection sets. 1: $\text{FIRST}(ABC) \setminus \varepsilon = \{a, b, c\}$, nullable so $+\text{FOLLOW}(S) = \{\$\}$. 2: $\{a\}$. 3: $\text{FOLLOW}(A) = \{b, c, \$\}$. 4: $\{b\}$. 5: $\text{FIRST}(C) \setminus \varepsilon = \{c\}$, nullable so $+\text{FOLLOW}(B) = \{c, \$\}$. 6: $\{c\}$. 7: $\text{FOLLOW}(C) = \{c, \$\}$.

(a)

	a	b	c	\$
S	1	1	1	1
A	2	3	3	3
B		4	5	5
C			**6, 7**	7

(b) Not LL(1): $M[C, c]$ holds 6 ($C \to c$) and 7 ($C \to \varepsilon$), a **FIRST/FOLLOW conflict**: $c \in \text{FIRST}(c)$ and $c \in \text{FOLLOW}(C)$. Sentence `c`: it can come from the last C ($A \to \varepsilon$, $B \to C \to \varepsilon$, $C \to c$) or from B ($B \to C \to c$, last $C \to \varepsilon$). Two trees: the grammar is ambiguous, so no table can work.

Gotcha

Use FIRST of the *right side*, not FIRST of the nonterminal: production 5 ($B \to C$) goes under c, never under b. And there is no ε column: ε-productions go under FOLLOW.

22 The LL(1) stack trace [mock 4.3, 5(b)]

Method

The idea. The stack holds what the parser still expects to read, top on the right. The parser either *expands* the nonterminal on top (using the table) or *matches* the terminal on top against the input.

1. **Set up** a table with columns: step, stack, remaining input, action. First row: stack $\$S$, input = the sentence followed by \$.
2. **Each row, look at the top X and the lookahead a:**
   - $X = a = \$$: write **accept** and stop.
   - X is a terminal: if $X = a$, write "match a"; next row: pop X and remove a from the input. If $X \neq a$: error.
   - X is a nonterminal: read $M[X, a]$. If it holds $X \to Y_1 Y_2 \cdots Y_k$, write its number; next row: pop X and push $Y_k, \ldots, Y_2, Y_1$, so that Y_1 is on top (written last, on the right). For $X \to \varepsilon$: just pop. The input does not change. Empty cell: error.
3. **Check the number of rows**: (productions used) + (tokens matched) + 1 (accept).

Mini-example. $S \to aS \mid b$, input `ab`: $\$S$ / `ab$`: $S \to aS$. $\$Sa$ / `ab$`: match a. $\$S$ / `b$`: $S \to b$. $\$b$ / `b$`: match b. \$ / `$`: accept. Rows: $2 + 2 + 1 = 5$.

Practice

Given. The list grammar of §17, numbered 1 $S \to (L)$, 2 $S \to a$, 3 $L \to SL'$, 4 $L' \to ,SL'$, 5 $L' \to \varepsilon$.

Your task. (a) Compute FIRST and FOLLOW and fill the LL(1) table. (b) Trace the parse of `(a,(a))`: one row per step with the stack, the remaining input, and the action.

Solution

(a) $\text{FIRST}(S) = \text{FIRST}(L) = \{(, a\}$, $\text{FIRST}(L') = \{,, \varepsilon\}$. $\text{FOLLOW}(S) = \{\$, ,,)\}$, $\text{FOLLOW}(L) = \text{FOLLOW}(L') = \{)\}$. Table: $M[S, (] = 1$, $M[S, a] = 2$, $M[L, (] = M[L, a] = 3$, $M[L', ,] = 4$, $M[L',)] = 5$; all other cells empty.

(b) Row check: productions used 1, 3, 2, 4, 1, 3, 2, 5, 5 (9), tokens matched 7, plus accept: 17 rows.

	Stack	Input	Action		Stack	Input	Action
1	$\$S$	`(a,(a))$`	1	10	$\$)L')L$	`a))$`	3
2	$\$)L($	`(a,(a))$`	match `(`	11	$\$)L')L'S$	`a))$`	2
3	$\$)L$	`a,(a))$`	3	12	$\$)L')L'a$	`a))$`	match `a`
4	$\$)L'S$	`a,(a))$`	2	13	$\$)L')L'$	`))$`	5
5	$\$)L'a$	`a,(a))$`	match `a`	14	$\$)L')$	`))$`	match `)`
6	$\$)L'$	`,(a))$`	4	15	$\$)L'$	`)$`	5
7	$\$)L'S,$	`,(a))$`	match `,`	16	$\$)$	`)$`	match `)`
8	$\$)L'S$	`(a))$`	1	17	\$	`$`	**accept**
9	$\$)L')L($	`(a))$`	match `(`				

Gotcha

Only a **match** shortens the input; an expansion never does. And push the right side reversed: for $S \to (L)$ the stack gets) L (with `(` on top.

23 Where is a syntax error caught? [mock 4.4]

Method

The idea. Both parsers reject the same inputs, but they may notice the error at different moments. The table parser checks the lookahead against the *exact* selection set every time it expands a nonterminal. Hand-written code often checks less.

1. **Table parser**: run the stack trace (§22) until it stops. Report the top of the stack X, the lookahead a, and why: the cell $M[X, a]$ is empty, or a terminal on top differs from a.
2. **Recursive-descent code**: simulate the calls (§17). In each procedure, test the lookahead against each branch condition *as written in the code*. Report the procedure and the call (`error()` in `T()`, or `match(SEMI)` called from `S()`).
3. **Compare.** The code can notice *later* for two reasons:
   - an ε branch written as a default ("otherwise return") accepts *any* token, not just FOLLOW; the error then surfaces at the next `match`;
   - a procedure with one alternative (`E(){ T(); Ep(); }`) checks nothing before calling, while the table has a row for E whose empty cells already reject.
4. **Answer format**: "table: at $M[X, a]$; code: in `...` called from `...`; the code is later because ...".

Practice

Given. The list grammar and table of §22, the erroneous input `(a a)`, and a recursive-descent parser whose `Lp()` is written *without* the check on `)`: `void Lp(){ if (la == ',') { match(','); S(); Lp(); } }` (otherwise it just returns).

Your task. (a) Trace the table parser until it stops: at which stack top and lookahead, and why? (b) Where does the recursive-descent parser report the error? (c) Explain the difference.

Solution

(a) \$$S$ / `(`: rule 1. Match `(`. \$)L / `a`: rule 3. \$)$L'S$ / `a`: rule 2. Match `a`. Now top L', lookahead `a`: $M[L', a]$ is **empty** (only `,` and `)` are allowed after a list element): error at L'.

(b) `S` matches `(`, calls `L`; `L` calls `S` (matches `a`) and `Lp`; `Lp` sees `a`, which is not `,`, and *returns*. Back in `S`, `match(')')` finds `a`: the error is reported in `match`, called from `S`.

(c) Same input rejected, but the code noticed one step later, because `Lp` accepted ε on any token instead of only on its selection set {`)`}.

Gotcha

Both parsers reject exactly the same inputs. The question is only *where* each notices; the table, with exact selection sets, never notices later than the code.

24 Removing immediate left recursion [mock 1.4(d), 3.1(b)]

Method

What the problem is. A production is *left-recursive* when its right side starts with its own left side: $E \to E + T$. A recursive-descent parser for E would call `E()` first thing, which calls `E()` again, forever (§17). So the grammar must be rewritten so that no rule starts with itself, *without changing the language.*

The idea. Look at what $E \to E + T \mid T$ actually generates: one T, then any number of $+T$:

$$T \quad T+T \quad T+T+T \quad \cdots \qquad \text{i.e.} \quad T\,(+T)^*.$$

The recursive rule only ever adds a *tail* ($+T$) on the right. So write the grammar the way the strings look: "a head, then a list of tails": $E \to T\,E'$ and $E' \to +T\,E' \mid \varepsilon$. The new nonterminal E' generates the list of tails, and its ε alternative ends the list.

1. **Sort the alternatives of A into two groups.** *Recursive* ones start with A: write each as $A \to A\,\alpha$ and keep only the part after A: that is a *tail* α. *Non-recursive* ones do not start with A: each is a *head* β. (If no alternative is non-recursive, the grammar generates nothing; something is wrong.)
2. **Write the new rule for A**: one alternative per head, each followed by the new nonterminal A':
$$A \to \beta_1\,A' \mid \beta_2\,A' \mid \cdots$$
3. **Write the rule for A'**: one alternative per tail, each followed by A' again, plus ε:
$$A' \to \alpha_1\,A' \mid \alpha_2\,A' \mid \cdots \mid \varepsilon$$
4. **Check** by generating a string from both grammars: the old one gives "one head then any sequence of tails"; the new one must give exactly the same. Then check for a terminal in each tail's first position (otherwise A' may have a FIRST/FIRST conflict).

Worked example. $E \to E + T \mid E - T \mid T$. Tails: $+T$ and $-T$. Head: T. New grammar: $E \to T\,E'$, $E' \to +T\,E' \mid -T\,E' \mid \varepsilon$. Check `T-T+T`: $E \Rightarrow TE' \Rightarrow T - TE' \Rightarrow T - T + TE' \Rightarrow T - T + T$. ✓

Special case: a list with ε. $L \to L\,X \mid \varepsilon$ ("zero or more X") has head ε and tail X. The recipe gives $L \to L'$, $L' \to X\,L' \mid \varepsilon$; since L is just L', drop the extra name and write $L \to X\,L \mid \varepsilon$. Same language, the recursion simply moved to the *right*.

Practice

Given. $R \to R\,a \mid R\,b\,c \mid d \mid e\,f$.

Your task. (a) Identify the tails (α's) and the heads (β's). (b) Write the grammar without left recursion. (c) Check that `dabc` is still generated.

Solution

(a) $\alpha_1 = a$, $\alpha_2 = bc$; $\beta_1 = d$, $\beta_2 = ef$.

(b) $R \to d\,R' \mid e\,f\,R'$, $\qquad R' \to a\,R' \mid b\,c\,R' \mid \varepsilon$.

(c) $R \Rightarrow dR' \Rightarrow daR' \Rightarrow dabcR' \Rightarrow dabc$. ✓ (Both grammars give one β followed by any sequence of α's.)

Gotcha

Every β must get the A' suffix, and A' must have its ε alternative. Forgetting either changes the language.

25 Removing indirect left recursion [course]

Method

1. Order the nonterminals $A_1, \ldots, A_n$.
2. For $i = 1, \ldots, n$: for every $j < i$, replace each production $A_i \to A_j\gamma$ by $A_i \to \delta\gamma$ for every production $A_j \to \delta$. Then remove the immediate left recursion of A_i (§24).
3. Rebuild FIRST/FOLLOW and the table: the result is not guaranteed to be LL(1).

Practice

Given. $A \to B\,a \mid c$, $\quad B \to A\,b \mid d$, $\quad$ start symbol A. ($A \Rightarrow Ba \Rightarrow Aba$: indirect left recursion.)

Your task. (a) Remove the left recursion using the order A, B. Is the result LL(1)? (b) Redo it with the order B, A. Is that result LL(1)? (c) Which version would you hand in, and what is the language?

Solution

(a) $i = 1$ (A): no $A \to A\ldots$, nothing to do. $i = 2$ (B): substitute A into $B \to A\,b$: $B \to B\,a\,b \mid c\,b \mid d$. Remove the immediate recursion: $B \to c\,b\,B' \mid d\,B'$, $B' \to a\,b\,B' \mid \varepsilon$. Not LL(1): FIRST$(Ba) = \{c, d\}$ meets FIRST(c), so $M[A, c]$ holds both A productions; and FOLLOW$(B') = \{a\}$ meets FIRST(abB'), so $M[B', a]$ is doubly filled.

(b) $i = 1$ (B): nothing. $i = 2$ (A): substitute B into $A \to B\,a$: $A \to A\,b\,a \mid d\,a \mid c$. Remove:

$$A \to d\,a\,A' \mid c\,A', \qquad A' \to b\,a\,A' \mid \varepsilon.$$

B is no longer reachable and can be dropped. Selection sets: $\{d\}$, $\{c\}$; $\{b\}$ and FOLLOW$(A') = \{\$\}$. **LL(1).**

(c) Version (b). The language is $(c \mid da)(ba)^*$.

Gotcha

Removing left recursion is necessary for LL(1), not sufficient, and the order of the nonterminals changes the result. Always rebuild the table before claiming LL(1).

26 Left factoring [course]

Method

1. For alternatives of A that start with the same symbols, take the *longest* common prefix α: $A \to \alpha\beta_1 \mid \alpha\beta_2 \mid \gamma$ becomes $A \to \alpha A' \mid \gamma$, $A' \to \beta_1 \mid \beta_2$.
2. Repeat on the new nonterminals until no two alternatives share a first symbol.
3. Check the table: factoring cannot remove ambiguity.

Practice

Given. (i) $A \to a\,b\,c \mid a\,b\,d \mid a\,e$. (ii) The if-statement grammar $S \to \mathsf{i}\,E\,\mathsf{t}\,S \mid \mathsf{i}\,E\,\mathsf{t}\,S\,\mathsf{e}\,S \mid \mathsf{a}$, $E \to \mathsf{b}$ (i = if, t = then, e = else).

Your task. (a) Left factor (i). (b) Left factor (ii). (c) Is the factored (ii) LL(1)? If not, which cell conflicts, and what does a parser do about it?

Solution

(a) Common prefix of all three: a. Then bc and bd share b:

$$A \to a\,A_1, \qquad A_1 \to b\,A_2 \mid e, \qquad A_2 \to c \mid d.$$

LL(1).

(b) Common prefix $\mathsf{i}\,E\,\mathsf{t}\,S$: $\quad S \to \mathsf{i}\,E\,\mathsf{t}\,S\,S' \mid \mathsf{a}, \quad S' \to \mathsf{e}\,S \mid \varepsilon, \quad E \to \mathsf{b}$.

(c) $\mathrm{FOLLOW}(S') = \mathrm{FOLLOW}(S)$, and $S' \to \mathsf{e}S$ puts e in $\mathrm{FOLLOW}(S)$. So $M[S', \mathsf{e}]$ holds both $S' \to \mathsf{e}S$ and $S' \to \varepsilon$: **not LL(1)**. The grammar is ambiguous (in $\mathsf{i\ b\ t\ i\ b\ t\ a\ e\ a}$, which i owns the e?). Parsers resolve it by keeping only $S' \to \mathsf{e}S$ in that cell: each else goes to the nearest if.

Gotcha

Factoring removes common prefixes, not ambiguity. If the conflict survives factoring, look for two parse trees.

27 Eliminating ε-productions [mock 1.6]

Method

The idea. If B can vanish, every rule that uses B must be able to do without it. So, instead of $B \to \varepsilon$, give each such rule a version where B is simply left out.

1. **Find the nullable nonterminals** (§18).
2. **For each production, list its nullable occurrences.** With k of them there are 2^k ways to keep or delete them. Write them all in a small table, then
   - drop the version with an empty right side (that would be a new ε-production),
   - drop duplicates and useless $A \to A$.
3. **Delete every ε-production.**
4. **If the start symbol S was nullable**, ε is in the language: add a new start symbol S' with $S' \to S \mid \varepsilon$.

Mini-example. $A \to B\,a\,B$, $B \to b \mid \varepsilon$. One production with two nullable occurrences:

first B	second B	version
keep	keep	BaB
delete	keep	aB
keep	delete	Ba
delete	delete	a

Result: $A \to BaB \mid aB \mid Ba \mid a$, $B \to b$. A is not nullable (it always has an a), so no new start symbol.

Practice

Given. $S \to A\,B \mid a$, $\quad A \to a\,A \mid \varepsilon$, $\quad B \to b\,B \mid \varepsilon$.

Your task. (a) Which nonterminals are nullable? (b) Give an equivalent grammar without ε-productions (except possibly $S' \to \varepsilon$ for a new start symbol).

Solution

(a) A, B (their ε-productions), then S (via $S \to AB$).

(b) Step 2, production by production:

Production	Nullable occurrences	Versions kept
$S \to AB$	A, B ($2^2 = 4$ versions)	AB, B, A (both deleted would be empty: dropped)
$S \to a$	none	a
$A \to aA$	A	aA, a
$B \to bB$	B	bB, b

Step 3: delete $A \to \varepsilon$ and $B \to \varepsilon$. Step 4: S was nullable, so add a new start:

$$S' \to S \mid \varepsilon, \qquad S \to A\,B \mid A \mid B \mid a, \qquad A \to a\,A \mid a, \qquad B \to b\,B \mid b.$$

Gotcha

With k nullable occurrences in one right side there are 2^k versions minus the empty one: the mock's $(S)S$ gives four. Forgetting $S' \to S \mid \varepsilon$ loses the empty string from the language.

28 Deciding "is this grammar LL(1)?" [mock 3.1(a), 5(a)]

Method

Check in this order; a "yes" at steps 1–3 already means "not LL(1)".

1. Is it ambiguous?
2. Is it left-recursive (directly or indirectly)?
3. Do two alternatives of one nonterminal have overlapping FIRST sets (e.g. a common prefix)?
4. For each nullable alternative of A: does FOLLOW(A) meet the FIRST of another alternative?

If all answers are no, it is LL(1). Justify a "no" with the cell and the two productions.

Practice

Given. Six grammars (start symbol S or E): (i) $S \to a\,S\,b \mid \varepsilon$; (ii) $S \to a\,S \mid a$; (iii) $S \to A\,a \mid b$, $A \to b \mid \varepsilon$; (iv) $S \to S\,a \mid b$; (v) $S \to a\,A \mid b$, $A \to c\,A \mid \varepsilon$; (vi) $E \to E + E \mid \mathsf{id}$.

Your task. For each: LL(1) or not, with a one-line reason.

Solution

	LL(1)?	Reason
(i)	yes	FIRST(aSb) = $\{a\}$, and $S \to \varepsilon$ is selected by FOLLOW(S) = $\{b, \$\}$: disjoint.
(ii)	no	common prefix a: both productions in $M[S, a]$. (Fix: $S \to aS'$, $S' \to S \mid \varepsilon$.)
(iii)	no	FIRST(Aa) = $\{b, a\}$ contains b, so $M[S, b]$ holds $S \to Aa$ and $S \to b$.
(iv)	no	left-recursive: FIRST(Sa) $\supseteq$ FIRST(b), both in $M[S, b]$.
(v)	yes	$A \to cA$ on $\{c\}$, $A \to \varepsilon$ on FOLLOW(A) = $\{\$\}$; S's alternatives start with a, b.
(vi)	no	ambiguous (id + id + id has two trees), and also left-recursive.

Gotcha

"Not LL(1)" describes the grammar, not the language: (ii) and (iv) have LL(1) grammars after factoring or removing left recursion. Only ambiguity cannot be fixed by these transformations.

29 Classifying a grammar and its language (Chomsky) [mock 1.1]

Method

The idea. There are two different questions: what type is *the grammar you are given*, and what type is *the language* (the simplest grammar that could exist for it). They can differ.

Type	Every production looks like	Recogniser	Example
3, regular	$A \to aB$, $A \to a$, $A \to \varepsilon$ (right-linear)	finite automaton	$(a \mid b)^*a$
2, context-free	$A \to \alpha$: one nonterminal on the left	pushdown automaton	$a^n b^n$
1, context-sensitive	$\alpha \to \beta$ with $\lvert\alpha\rvert \le \lvert\beta\rvert$	linear-bounded automaton	$a^n b^n c^n$
0, unrestricted	$\alpha \to \beta$	Turing machine	—

1. **Type of the grammar**: check every production against the rows, starting from type 3. The grammar's type is the most restrictive row that *all* its productions satisfy.
2. **Type of the language**: describe the language in words or as a set first. Then
   - if you can write a regular expression or a DFA for it: regular (type 3), whatever the given grammar is;
   - if not, prove it is not regular with distinguishable strings (§10); if a CFG generates it, it is context-free (type 2) and not regular.
3. **Answer format**: "grammar: type X because production ... ; language: ... , type Y because ...".

Practice

Given. (a) $S \to a\,S \mid b\,S \mid a$; (b) $S \to a\,S\,b \mid \varepsilon$; (c) $S \to a\,S \mid S\,b \mid \varepsilon$.

Your task. For each, give the type of the grammar and the type of the language it generates, with a reason.

Solution

(a) Grammar type 3 (every production is $A \to aB$ or $A \to a$). Language $(a \mid b)^*a$: regular.

(b) Grammar type 2 (aSb is not right-linear). Language $\{a^n b^n\}$: not regular (a^i and a^j are distinguished by b^i), so context-free and not regular.

(c) Grammar type 2 ($S \to Sb$ is not right-linear). But the language is a^*b^*, which is **regular**: a type-3 grammar exists ($S \to aS \mid B$, $B \to bB \mid \varepsilon$).

Gotcha

The type of the grammar you are given is not the type of the language. (c) has a context-free grammar for a regular language.